This assignment does not count toward the final grade.

Lab 9

- Due No Due Date
- Points None

Lab 9: The Image Sensor Control Plane

The CMV300 on your board is a 648 × 488 global-shutter imager. This week you will not take a picture with it. You will make it run, and then measure everything it does, without storing a single pixel.

That ordering is deliberate. A camera that produces a wrong image gives you one symptom and about ten possible causes. By the end of this lab you will have proved, with numbers, that the sensor is clocked correctly, that it is producing exactly 488 rows of exactly 648 pixels, that its frame timing obeys the datasheet formula in both of its operating regimes, and that all ten data lines are alive. Then when the first image looks wrong, the sensor is not on the list of suspects.

Everything here is measured with counters

There is no oscilloscope on the video signals and no frame buffer. Some of the counters are given to you and some you write. Every acceptance number in this lab is a number read back over FrontPanel from a counter inside the FPGA.

Building instruments into a design, rather than reaching for a bench instrument, is how you measure things that happen inside a chip.

What Is Due, and When

Item	Points	Due
Lab 9 – Pre Lab (individual)	15	Start of your lab session
Lab 9 – Milestones (in pairs, demonstrated)	40	First 60 minutes of your next lab session
Lab 9 – Post Lab (individual)	45	First 60 minutes of your next lab session
Total	**100**	

The demonstration and the written work are due at the same moment: the first 60 minutes of the following session. **Late submission costs 10% per day**.

Files for This Lab

Folder	Contents
`lab9/code`	`SensorBoard_CMV300.zip` (https://canvas.illinois.edu/courses/74846/files/23850929?wrap=1) (https://canvas.illinois.edu/courses/74846/files/23850929/download?download_frd=1) — the project, the simulation models, the testbench and the Python API
`lab9/reference`	CMV300 datasheet (ams v2.11), sensor board schematic
`Shared Files/reference`	Opal Kelly FrontPanel documentation

The package builds as shipped and its testbench runs as shipped. Your work replaces three stubs inside it.

What Is Given, and What You Write

The camera subsystem is a set of modules with a fixed interface. Three of them are stubs.

Module	Status
`CMV300_Ctrl.v`	**you write it** — power-up sequencer, reset, FRAME_REQ pulse generator
`CMV300_Spi.v`	**you write it** — the 16-bit SPI master
`CMV300_Counters.v`	**you write it** — the row and pixel counters, and the OR/AND pixel masks
`CMV300_Meters.v`	given — the CLK_OUT meter and the frame-timing meter
`CMV300_Subsystem.v`	given — wrapper, endpoint map, clock generation
`SensorBoard_Top.v`	given — do not edit

`sim/cmv300_model.v`	given — a behavioural sensor: SPI slave with the datasheet defaults, plus a video generator with the real row and frame timing
`sim/tb_lab9.v`	given — the testbench that decides whether you are finished

Why that split, and not a different one

The two given meters measure the sensor's own clocking. They do not depend on anything you build, and you need them working from the first minute to debug everything else — the CLK_OUT meter in particular is how you will discover that the datasheet is wrong about one register.

The counters you write sit on the **pixel bus**, and there the method genuinely matters. A row is not 648 back-to-back valid pixels, and a counter that assumes it is will give you a confident wrong answer. That is the part worth doing yourself, and it is the front end of the capture logic you will write next week.

> ### The interface is the specification, and it is frozen.
>
> The port lists of the three stubs are fixed. If you change one, the given modules stop connecting and the testbench stops compiling. Everything you need to know about what your modules must do is in those port lists plus the named checks in `sim/tb_lab9.v`, and both are written out in `INTERFACE.md`.
>
> When you ask a language model for help, **paste the port list into the prompt.** A module that does the right thing with the wrong interface is worth nothing here.

Work in Simulation, Build Once

A full build of this project takes about **15 minutes**. A simulation run takes seconds.

```
python build.py --sim     # the testbench, seconds
python build.py           # the bitstream, ~15 minutes
```

The same two commands on every machine

`build.py` is a Python script, not a shell script, so it runs identically in the Windows Command Prompt, in PowerShell, and in a Mac or Linux terminal. You already need Python for everything else in this course, so there is nothing extra to install.

It needs Vivado on your PATH. If it is not, the script says so and tells you what to add rather than failing somewhere deep in a build.

So the loop is: write, simulate, read the failure, fix, simulate. Only when `python build.py --sim` prints `RESULT: ALL PASS` do you start a build. The behavioural sensor model responds to SPI, generates video with the correct row and frame timing, and is good enough that every acceptance number in this lab can be reached before you touch hardware.

What to paste when you ask for help

The testbench prints a named check and a simulation time when it fails: `FAIL scenario 3 check 7 at 41250 ns: expected 488 rows, got 487`. That line, the port list from `INTERFACE.md`, and the relevant datasheet figure are all a language model needs.

A 500-line Vivado log is not. If the answer depends on something you cannot paste into the chat, it is not a question to ask a model — it is a question to answer with the testbench.

The Sensor as an SPI Peripheral

Before it is a camera, the CMV300 is a register file behind a four-wire SPI port. Datasheet §3.9.

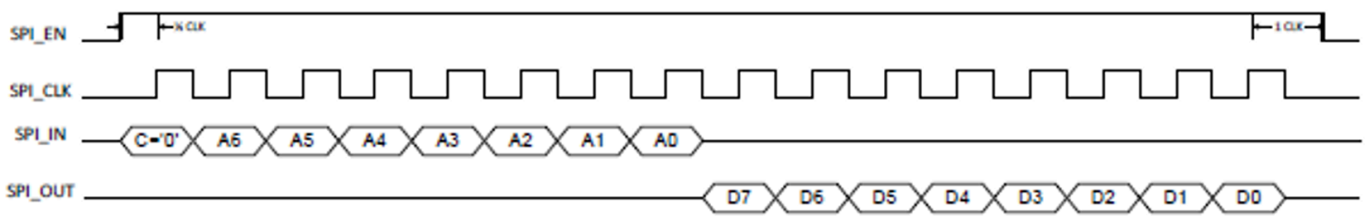


Figure 1. The SPI frame. 16 bits, MSB first: one control bit, seven address bits, eight data bits.

Field	Bits	Meaning
Control	15	1 = write, 0 = read
Address	14:8	register number, 0 to 127
Data	7:0	the value written, or ignored on a read

The sensor **samples SPI_IN on the rising edge** of SPI_CLK and **launches SPI_OUT on the falling edge**. Run the clock at 10 MHz or below. Test points **TP1 to TP6** bring the SPI signals out, so you can put a scope on a transaction and check it against the datasheet figure.

Two registers make one number

Several sensor parameters are wider than eight bits and are split across two registers, low byte first. `reg 1` and `reg 2` together are Number_lines, and `reg 42` with `reg 43` is Exp_time.

So a correct read of reg 1 alone tells you almost nothing. The known answer is the *composite*: 232 + 256 × 1 = **488**, which is the number of rows the sensor has. If your reconstruction gives something else, you have found either a byte-order error or an off-by-one in your shift register before it could hide inside an image.

Power-Up Is a Sequence, Not a Switch

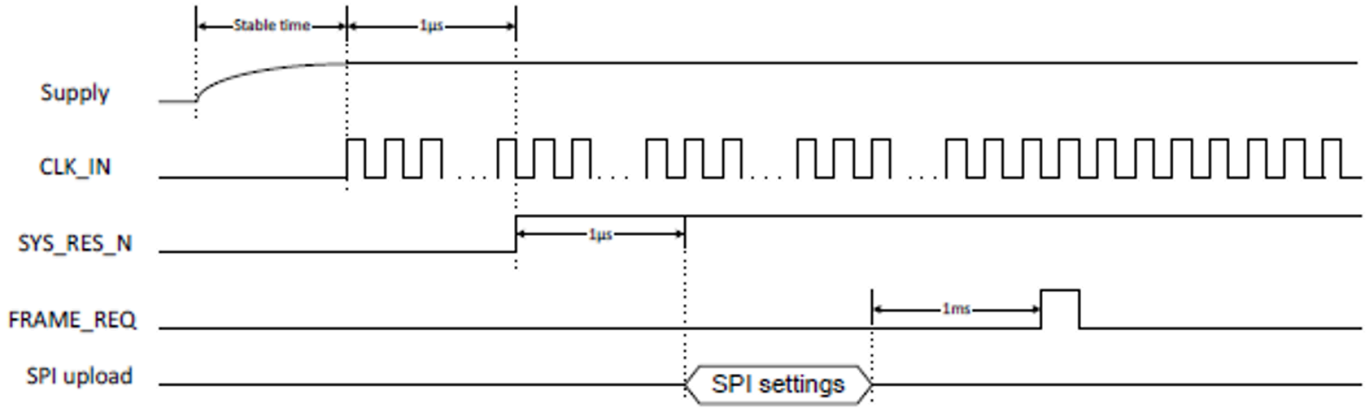


Figure 2. Datasheet Fig. 5. Supplies, then clock, then reset release, then SPI.

1. Supplies stable.
2. **CLK_IN running**, then wait at least **1 µs**.
3. Release **SYS_RES_N**, then wait at least **1 µs**.
4. SPI is now allowed. Upload your register configuration.
5. Wait **1 ms** for the PLL to lock before the first FRAME_REQ.

Get the order wrong and there is no error, no status bit and no complaint. The sensor simply does not respond, or responds with values that make no sense. Your sequencer must enforce this in hardware, because the host cannot time 1 µs reliably.

The Clock, and a Register the Datasheet Gets Wrong

Two registers stand between you and a running sensor clock.

Register	Value	What it does
`reg 69`	9	routes CLK_OUT to its pin. Until this is set, **there is no output clock at all** and the meter reads zero. An older datasheet edition says 8; that was a typo, corrected in the v2.11 change record.
`reg 83`	251	the PLL frequency band. The bands are 251 for 10–32 MHz, 187 for 20–50 MHz, and 155 for 32–50 MHz.
`reg 57`	3	Output_mode: the parallel port.

> The datasheet default for reg 83 is wrong for this board.
>
> The power-on default is **155**, which is the 32–50 MHz band. Your CLK_IN is 25 MHz, which is in the 10–32 MHz band, so the correct value is **251**.
>
> With the default the PLL still locks. It locks to the wrong frequency, and the CLK_OUT meter reads about **36,700 cycles per millisecond** instead of 25,000. Nothing else complains. Every timing measurement you make afterwards is wrong by 47%, and every image you eventually capture looks fine.

This is why the meter exists and why you read it before anything else. A frequency you have measured is a known answer; a frequency you assumed is a hope.

FRAME_REQ: The First Command With No Reply

Everything you have sent a peripheral so far came back with something. An I²C read is acknowledged and returns bytes. An SPI read returns the register. A SCPI query returns a string. Even the ninth clock in Lab 5 was a bit the sensor drove, so silence was itself the diagnosis.

FRAME_REQ returns nothing. You pulse a pin and the sensor goes quiet for several milliseconds. There is no status bit that says it is working, no error if it did not happen, and nothing to read back. The only evidence is that video signals begin appearing on entirely different pins, some milliseconds later.

So verification changes shape

You stop checking the reply and start checking the **consequence**, and the consequence is a timing pattern: how long the silence lasted, how many LVAL pulses followed, how many DVAL cycles sat inside each one.

This is the same move as arming an oscilloscope trigger rather than reading a meter.

The contract

A pulse of at least one CLK_IN period requests `Number_frames` frames (registers 55 and 56). Exposure starts, runs for the programmed time, then the frame overhead time (FOT) transfers the image to the readout registers, then the sensor reads out automatically. Nothing you do in between has any effect.

FRAME_REQ must be a pulse

Held high, it re-triggers continuously and you get frames forever. That is useful later, but it is not what you want when you are timing one frame. Your sequencer generates a pulse of a few CLK_IN periods from a single trigger.

The sensor has two timing regimes, and they give different answers

This is the part that catches people, because the same registers produce two different frame rates depending on how you ask.

	One frame, requested from idle	**Pipelined (Number_frames > 1, or free-run)**
What happens	exposure, then FOT, then readout, strictly in sequence	the **next exposure overlaps the current readout**
Silence after FRAME_REQ	**6.55 ms** at the default exposure, before the first pixel	none after the first frame; rows arrive continuously
Readout duration	12.69 ms of streaming	the same 12.69 ms, but it is happening while the next frame exposes
Frame-to-frame period	**19.24 ms** = 52 fps	**12.88 ms** = 77.6 fps
Governed by	exposure + FOT + readout, added up	FOT + max(readout, exposure)

A 33% throughput difference, for free

Identical registers, identical sensor, identical everything except whether you asked for one frame or many. Pipelining does not make the readout faster; it stops you paying for the exposure separately.

And notice that the two regimes answer different questions. The pipelined period tells you how *often* a frame arrives. The single-frame time tells you how *soon* you can have one after asking. Those are not the same number and later in this course you will care about both.

Five ways to get nothing, none of which reports an error

Precondition	Symptom if wrong
CLK_IN running and the PLL band correct	CLK_OUT meter reads 0, or reads 36,700 instead of 25,000
`reg 69` = 9	CLK_OUT meter reads 0; no video signals at all
`Number_frames` ≥ 1	the pulse is accepted and nothing happens. The minimum is 1; there is no 0 = continuous
SYS_RES_N released at least 1 µs after the clock started	SPI reads return nonsense, or the sensor never becomes ready
at least 1 ms elapsed since the SPI upload	the PLL has not locked; the first frame is missing or malformed

Milestone 3 asks you to trigger three of these deliberately and record what the counters say.

What the Sensor Emits, and How to Count It

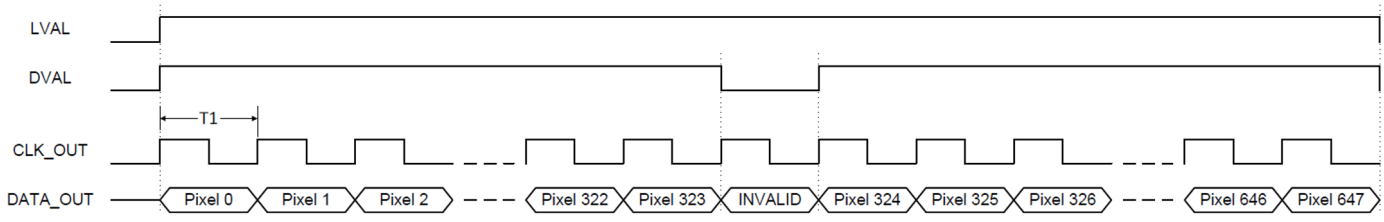


FIGURE 26: PARALLEL OUTPUT TIMING OF ONE LINE

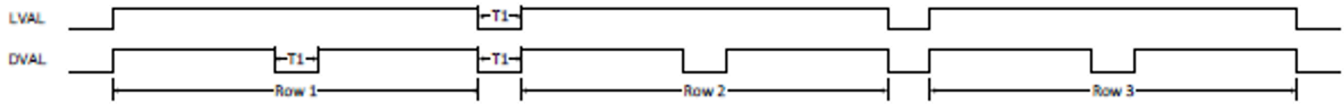


FIGURE 27: LVAL/DVAL TIMING FOR A FRAME OF 3 LINES USING THE PARALLEL OUTPUT

Figure 3. Datasheet Figs. 26 and 27. One row: 324 pixels, one invalid slot, 324 pixels.

Sample D[9:0], LVAL and DVAL on the **falling edge** of CLK_OUT (§4.2.1). Then:

- **LVAL** is high for one row. Count its rising edges over a frame and you have the number of rows.
- **DVAL** is high when the pixel on D[9:0] is real. Count DVAL-high CLK_OUT cycles within one LVAL and you have the pixels per row.
- The row is **not** 648 contiguous pixels. It is 324, then one slot with DVAL low, then 324. A counter that assumes 648 back-to-back valid pixels will be off by one from the middle of every row onward.

> That single invalid slot is the kind of detail that costs an afternoon in the next lab and thirty seconds in this one, because here you are counting rather than looking at an image. It is also the reason these counters are yours to write rather than given to you.

What You Upload

Two Canvas assignments, and the code and the report go to different ones.

Assignment	What goes there	When
Lab 9 – Milestones	`lab9_CMV300_Ctrl.v`, `lab9_CMV300_Spi.v` and `lab9_CMV300_Counters.v` as three separate files	**before your demonstration**
Lab 9 – Post Lab	`lab9_postlab.pdf`	by the same deadline

> The code has to be uploaded before the demo, not after.
>
> The TA checks Canvas before starting and will not begin until your three files are showing as submitted. A demonstration with no code submitted scores **zero** for the milestone, whatever the board did.

Next

The **Lab 9 – Pre Lab** is due before your session, and question 2 is the arithmetic every measurement in the lab is checked against. The build is on **Lab 9 – Milestones**.